

Dataset Generation

(scripts/generate_dataset.py)

Why Generate Data This Way

The neural solver needs thousands of (`particle_state_t`, `acceleration_t`) pairs to train on. All three learned simulation frameworks [[paper7](#), [paper9](#), [paper10](#)] use this same paradigm: run a reference physics simulator, record full trajectories, and train a network to predict the per-particle acceleration from the current state.

[[paper7](#)] generates data with Taichi MPM on GPU (128×128 grid, 4k particles, 1k trajectories per scenario). Our setup mirrors this: Taichi MLS-MPM [[paper2](#)] on a 64×64 cloth with target 10k+ total clips.

2 Three Scenario Families

Scenario	What varies	Pinned corners	Wind
<code>drape</code>	Initial height (0.6–0.9 m), sphere position (± 15 cm), sphere radius (0.15–0.25 m)	None	None
<code>wind</code>	Wind speed (0.5–4 m/s), direction (0–360°)	Top two corners [0, 63]	Stored in metadata (M2 smoke: not yet applied)
<code>collision</code>	Sphere position (± 15 cm), sphere height (0.2–0.4 m), radius (0.10–0.18 m)	Top two corners	None

The diversity of initial conditions across scenarios is motivated by [[paper7](#)]'s multi-scenario datasets (Water, WaterRamps, Sand, SandRamps, etc.) and [[paper9](#)]'s generalization experiments — coverage of varied dynamics is essential for a robust neural solver.

3 Clip Format

Each clip is one `.npz` file with these arrays:

Key	Shape	What it is
<code>x</code>	<code>(T, N, 3)</code>	Particle positions at each logged frame
<code>v</code>	<code>(T, N, 3)</code>	Velocities
<code>a</code>	<code>(T, N, 3)</code>	Accelerations — the M3 training label
<code>F</code>	<code>(T, N, 3, 3)</code>	Deformation gradients — input feature for M3
<code>contact_flag</code>	<code>(T, N)</code>	Whether each particle touches the sphere
<code>meta</code>	scalar object	Scenario name, seed, all parameters, config hash

The choice to save accelerations as the training target follows all three reference papers: [\[paper7\]](#) predicts $\ddot{a}_p$, [\[paper9\]](#) predicts $\ddot{x}_i$, and [\[paper10\]](#) predicts $p_i = \ddot{x}_i$. Semi-implicit Euler integration $v^{t+1} = v^t + dt * a$, $x^{t+1} = x^t + dt * v^{t+1}$ recovers positions [\[paper9, §4.2\]](#).

The particle state $\{x, v, F\}$ matches the training-data state recommended in [\[paper2, §7\]](#): expose $\{x_p, v_p, F_p, C_p, m_p, \sigma_p\}$ as the inputs for any neural grid-update replacement.

T = number of logged frames. With $dt = 1e-4$ s and `log_every = 10`, each logged frame covers 1 ms of simulated time.

4 Determinism and Reproducibility

Every clip can be regenerated bit-exactly from its `(scenario, seed)` pair:

```
seed = seed_base[scenario] + i # e.g., drape seed 0, wind seed 100000
```

```
rng = np.random.default_rng(seed)
```

```
spec = sample_drape_clip(rng, base_cfg, seed) # deterministic parameters
```

```
clip = run_one_clip(spec, base_cfg, smoke=False) # deterministic sim
```

The seed is baked into the Taichi init (`random_seed=seed`) and the config hash is stored in the metadata, making the exact data reproducible even after code changes.

5 Seed Partitioning Prevents Leakage

Train : seeds [0, 1_000_000) drape: 0..3999, wind: 100000..102999, ...

Val : seeds [1M, 2_000_000)

Eval : seeds [2M, 3_000_000) frozen; never loaded during training

The 15 frozen eval clips (5 seeds × 3 scenarios) are never seen during training or detector calibration. This mirrors the train/val/test split used in [paper10] (1000 train / 100 val / 100 test) and [paper7]'s held-out test trajectories.